

TD6 : Les fichiers

Xavier André & Romain Tavenard

Ce sujet traite des fichiers JavaScript Object Notation (JSON). Ce format permet de stocker des données structurées, par exemple avec une organisation hiérarchique. Dans ce TD, on se tourne vers un jeu de données recensant des notes attribuées à des restaurants de la ville de New York.

Pour ce TD, vous écrirez votre code dans un fichier `td6.py` dont la structure sera la suivante :

```
# Section 1 : Imports de module

# Section 2 : Définition de fonctions

# Section 3 : Tests de fonctions définies et manipulations en mode "script"
"
if __name__ == "__main__":
    pass # Remplacer par vos appels de test
```

Bonne pratique : tous vos appels de test (section 3) doivent se trouver à l'intérieur du bloc `if __name__ == "__main__":`. Cela garantit qu'ils ne s'exécutent pas si votre fichier est importé depuis un autre script.

1 Chargement des données

1. Sur CURSUS, télécharger le jeu de données `NYfood.json` et l'enregistrer dans un dossier `data` de votre dossier de travail dans VS Code.
2. Visualiser avec Visual Studio Code le contenu du fichier JSON.
3. Quels sont les attributs (clés de dictionnaires) de ce jeu de données ? Que contient l'attribut `grades` (qui signifie `notes` en français) ?

2 Extraction d'informations élémentaires

Dans la suite de ce sujet, on nomme “restaurant” chaque dictionnaire issu du fichier JSON. Les questions suivantes nécessitent de manipuler les données depuis Python et le code produit devra être inclus dans un nouveau fichier `td6.py`.

1. Charger le contenu du fichier `NYfood.json` dans une liste de dictionnaires, chaque dictionnaire représentant un restaurant. Combien y a-t-il de restaurants dans ce fichier ?
2. Combien y a-t-il de types de cuisine (`cuisine`) différents dans ce fichier ? Afficher la liste des types de cuisine présents.
3. Écrire une fonction qui prend en entrée la liste des restaurants et retourne une liste des noms de restaurants ayant reçu au moins une note “A”.
4. Écrire une fonction qui prend en entrée la liste des restaurants et affiche, pour chaque restaurant dont le nom commence par la lettre “S”, son nom et son type de cuisine.
5. Quel est le nombre total de restaurants dont le code postal (`zipcode`) commence par “10” ?
6. Quelles sont les valeurs possibles du sous-attribut `date` de l'attribut `grades` (date d'attribution de la note) pour les restaurants ayant reçu une note “B” ?
7. Écrire une fonction qui prend en entrée la liste des restaurants et retourne un dictionnaire associant chaque type de cuisine au nombre de restaurants correspondant.

3 Export au format CSV

Dans cette partie, il va s'agir d'enregistrer un export du contenu du fichier `NYfood.json` au format CSV, en s'assurant donc que l'on n'ait plus d'attributs imbriqués.

1. Écrire une fonction qui prend en entrée le jeu de données précédent et retourne une nouvelle version de ce jeu de données dans lequel l'attribut `grades` a été supprimé et remplacé par un attribut `n_grades` qui indique le nombre de notes reçues.
2. Écrire une fonction qui prend en entrée le jeu de données précédent et retourne une nouvelle version de ce jeu de données dans lequel l'attribut `address` est supprimé et remplacé par deux nouveaux attributs `latitude` et `longitude` (l'attribut `coordinates` est une liste contenant la longitude et la latitude, stockées dans cet ordre).
3. En utilisant les deux fonctions précédentes, exporter cette nouvelle version du jeu de données dans un fichier nommé `NYfood.csv`.